

SHL Assessment Recommender — Approach Document

1. Design Choices

Architecture: Stateless FastAPI Service

Every POST /chat call carries the full conversation history. The handler reconstructs all context (constraints, prior shortlists, conversation state) purely from the message array on every call. No server-side session storage, no database — the function signature is effectively $f(\text{messages}) \rightarrow \text{response}$.

Why: The grading harness sends no session ID. Statelessness also simplifies deployment (no shared state between instances) and makes the service trivially horizontally scalable.

LLM: Groq + llama-3.3-70b-versatile

Groq was chosen for its sub-second inference latency on large models, which leaves ample headroom inside the 30-second per-call budget for retrieval + response assembly. The llama-3.3-70b-versatile model provides strong instruction-following for structured JSON output without sacrificing speed.

Critical design split: The LLM handles only (1) intent/constraint extraction from conversation and (2) natural-language reply phrasing. It never decides which catalog items exist — that is strictly the retrieval layer's job. This separation protects the hard-eval schema compliance requirement from LLM hallucination.

Async Handlers

All FastAPI endpoints and Groq calls use `async/await` (AsyncGroq). This prevents blocking under concurrent requests from the grading harness.

2. Retrieval Setup

Catalog Sourcing

The catalog was ingested from `https://tcp-us-prod-rnd.shl.com/voiceRater/shl-ai-hiring/shl_product_catalog.json` — a flat JSON array of 377 items. Downloaded once at build time and saved as `data/catalog.json`.

Individual Test Solutions vs. Job Solutions Filtering

Heuristic: Exclude items whose name contains "Solution" (case-sensitive). This identified 7 bundled products (e.g., "Customer Service Phone Solution," "Entry Level Cashier Solution") that match the definition of pre-packaged Job Solutions — they combine multiple assessment types for specific job families.

Reasoning: All 7 excluded items have names ending in "Solution" and their keys arrays span multiple disparate categories bundled together. Individual Test Solutions, by contrast, are named after specific skills/constructs (e.g., "Python 3 (New)," "Apache Kafka (New)") and typically have 1-2 closely related keys. No items appearing in the gold trace shortlists were excluded by this filter — cross-checked against all 10 traces.

Result: 370 items retained after filtering.

Retrieval Approach: TF-IDF + Keyword Matching (Hybrid)

Built a `TfidfVectorizer` (scikit-learn) over the catalog at startup. Each document = concatenation of name + description + keys + job_levels + languages. Unigram + bigram features with sublinear TF for better phrase matching.

Why not embeddings: The catalog is small (~370 items). TF-IDF over keyword-heavy technical terms (programming languages, test names) performs well for exact and near-exact matching. It avoids downloading a sentence-transformer model (which would slow startup and risk blowing the 2-minute cold-start window on free-tier hosts).

Soft filters: Job level and test type are applied as score multipliers (not hard exclusions) to avoid over-filtering when the catalog is already small.

OPQ32r soft prior: Per §10c, the traces nearly always include OPQ32r as a default personality measure in hiring contexts. The retrieval layer boosts it into the candidate set when the query context suggests general hiring/selection.

3. test_type Derivation

The response's `test_type` field is derived as a **deterministic full join** of each item's keys array, mapped through the letter legend: A (Ability & Aptitude), B (Biodata & SJ), C (Competencies), D (Development & 360), E (Assessment Exercises), K (Knowledge & Skills), P (Personality & Behavior), S (Simulations). Letters are sorted alphabetically and comma-joined.

Decision rationale: This is the dominant pattern observed across the 10 traces. One trace showed a single-letter `test_type` for a 6-category item, and another showed a likely typo (K paired with "Simulations"). Rather than reverse-engineering undocumented "primary category" logic from these inconsistencies, the full-join approach is defensible, deterministic, and matches the majority of trace examples.

4. Prompt Design

The LLM is called with a structured intent-extraction prompt that returns JSON with: `intent` (clarify/recommend/refine/compare/refuse), `constraints` (role, seniority, skills, test types, etc.), `draft_reply`, `additions/removals` (for refine), and `compare_items`.

Why not trust the LLM to emit the final response schema directly: JSON-mode flakiness is a real risk with any LLM. The hard-eval schema compliance requirement means a single malformed response could fail the entire submission. By having the LLM return an intermediate structure and assembling the final `reply/recommendations/end_of_conversation` JSON in Python via Pydantic, we get compile-time-like guarantees on the wire format.

Refusal detection: The prompt explicitly lists categories of off-topic requests (hiring advice, legal questions, prompt injection) and maps them to a `refuse` intent. The Python code then returns a schema-valid refusal response — this is not left to "hope the prompt handles it."

5. Evaluation Approach

Local Harness

Built a trace-replay harness (`tests/eval_harness.py`) that: (1) parses each .md trace file to extract user messages and the expected final shortlist, (2) deduplicates traces by content hash, (3) replays user messages turn-by-turn against the running /chat endpoint, and (4) computes Recall@10 per trace and the mean across all traces.

Behavior Probes (tests/test_behaviors.py)

16 binary pass/fail tests covering: - Vague turn-1 → must clarify (not recommend) - Specific turn-1 → may recommend immediately - Off-topic / legal / salary / prompt injection → must refuse - Multi-fact dump → handle gracefully - Correction → update, not restart - URL hallucination check → 0% tolerance

Schema Compliance (tests/test_schema.py)

8 tests verifying the exact response shape on every code path: empty messages, vague messages, specific messages, malformed requests (invalid role, empty content), recommendations always a list (never null), end_of_conversation always a boolean, no extra fields.

6. What Didn't Work

Initial approach: single LLM call for everything. The first iteration had the LLM both extract intent and generate the full response JSON (including recommendation names/URLs). This caused two problems: (1) the LLM would hallucinate assessment names not in the catalog, and (2) occasional JSON formatting errors broke schema compliance. Splitting into LLM-for-intent + retrieval-for-items + Python-for-assembly eliminated both issues.

Overly aggressive clarification. An early version asked 3-4 clarifying questions before ever recommending, which burned through the 8-turn budget. Tuning the prompt to recognize sufficient signal on turn 1 (specific skills, test types, detailed JD) and limiting clarification to genuinely ambiguous queries fixed this.

7. AI Tool Usage

This project was developed with assistance from an AI coding agent (Claude in an IDE agent context). The agent was used for: initial architecture planning, generating boilerplate code structure, writing the system prompts, creating test scaffolding, and iterating on the retrieval approach. All code was reviewed, tested, and modified as needed. The catalog data pipeline, filtering heuristics, and evaluation harness logic were designed through collaborative iteration with the AI tool.